Davante

17

BASE DE DATOS

Subprogramas y disparadores

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Procedimientos 5

/ 3. Funciones 6

/ 4. Paquetes 7

/ 5. Disparadores o triggers 8

5.1. Concepto de disparador 8

5.2. Creación de disparadores 8

/ 6. Explotación de disparadores: Restricciones, orden y gestión 9

6.1. Restricciones en los disparadores 9

6.2. Orden de ejecución de los disparadores 9

6.3. Borrado y activación o desactivación de disparadores 10

/ 7. Explotación de disparadores: Referencias y condicionales 10

7.1. Referencias NEW y OLD 10

7.2. Cláusulas IF INSERTING, IF UPDATING e IF DELETING 11

/ 8. Explotación de los disparadores: Tipos especiales e información adicional 11

8.1. Disparadores de tipo INSTEAD OF 11

8.2. Información sobre disparadores 12

ÍNDICE

/ 9. Automatización de la ejecución de scripts 12

9.1. Oracle Enterprise Manager (OEM) 12

9.2. DBMS_JOB 12

9.3. Shell Scripts con SQL*Plus 13

9.4. Cron Jobs (Linux/Unix) 13

/ 10. Automatización de la ejecución de scripts: Oracle Scheduler 13

10.1. Creación de jobs 13

10.2. Modificación y eliminación de jobs 14

10.3. Comandos adicionales 14

/ 11. API de acceso a bases de datos 14

/ 12. Caso práctico 1: “Procedimientos y funciones más complejas” 15

/ 13. Caso práctico 2: “Creación de disparadores” 16

/ 14. Resumen y resolución del caso práctico de la unidad 17

/ 15. Bibliografía 18

© MEDAC ISBN: 978-84-19885-68-5

Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Explorar las distintas clases de subprogramas disponibles en PL/SQL con el fin de mejorar el dominio y aprovechar al máximo las capacidades de este lenguaje de programación.

Entender el concepto de trigger o disparador, además de aprender a utilizarlo y aplicarlo eficazmente.

Aprender a comprender y utilizar las API en PL/SQL para acceder a la información almacenada en una base de datos.

Analizar las diversas formas de automatizar la ejecución de scripts.

/ 1. Introducción y contextualización práctica

El concepto de subprograma, que abarca tanto funciones como procedimientos, está vinculado al nivel de abstracción que proporciona un lenguaje de programación. Esto permite la creación de unidades de código con funcionalidades específicas, que pueden ser invocadas en otras secciones del programa principal.



Fig.1. Oracle dispone de un catálogo de API actualizado.

En este tema profundizaremos en la naturaleza de los procedimientos, funciones y paquetes, así como en su correcta aplicación en PL/SQL.

Además, exploraremos el concepto de disparador (*trigger*), una poderosa herramienta en PL/SQL para la programación de bases de datos, que está estrechamente relacionada con los procedimientos.

También analizaremos las diversas formas de automatizar la ejecución de *scripts*, optimizando así el manejo y la eficiencia del código.

Finalmente, examinaremos diversas API para el acceso a bases de datos utilizando lenguajes de programación externos. Estas API son cada vez más comunes debido a la necesidad de implementar accesos a bases de datos en distintos entornos y utilizando diversas tecnologías, como conexiones desde la web, JAVA, C, entre otros.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Soluciones en PL/SQL para la gestión, el mantenimiento y la automatización en la BD"

https://on.soundcloud.com/DoiWV9kcLQ8hHKcy6

/ 2. Procedimientos

Un procedimiento, en esencia, es un **bloque de código diseñado con un propósito específico**, al cual **se le asigna un nombre para** poder **reutilizarlo en distintas partes del programa**. La **sintaxis** para **declararlo** se muestra en la figura 2.

```
CREATE [OR REPLACE] PROCEDURE nombreProcedimiento
[(variable1 [modelo] tipoDatos, variable2 [modelo] tipoDatos…)]
{IS|AS}
Declaraciones
BEGIN …
```

Fig.2. Sintaxis de declaración de un procedimiento.

La opción **REPLACE** permite **sustituir un procedimiento existente con el mismo nombre por el que se acaba de crear**. Asimismo, el parámetro 'modelo' define el tipo de variable. Escucha el siguiente audio para obtener más detalles al respecto:

Audio 1. "Parámetros en procedimientos y funciones"
https://on.soundcloud.com/TADoRTe4DxuZEzNw6

Los procedimientos se pueden **invocar** desde otros bloques de código simplemente **usando su nombre**. En la figura 3, se presenta un ejemplo donde se llama a un procedimiento denominado 'consultarEmpleado', que recibe información sobre empleados a través de parámetros:

```
DECLARE
    nom VARCHAR2(20);
    sal NUMBER;
BEGIN
    consultarEmpleado(1,nom,sal);
    DBMS_OUTPUT.PUT_LINE(nom);
    DBMS_OUTPUT.PUT_LINE(sal);
END;
```

Fig.3. Ejemplo de llamada a un procedimiento llamado 'consultarEmpleado'.

También es posible **invocar procedimientos desde SQL Plus** utilizando la sintaxis de la figura 4.

```
EXECUTE nombreProcedimiento (parámetros);
```

Fig.4. Sintaxis para invocar un procedimiento.

Si es necesario **eliminar un procedimiento**, se puede hacer empleando el comando **DROP PROCEDURE** seguido del nombre del procedimiento.

Finalmente, en Oracle, es posible **revisar el código fuente** de procedimientos y funciones utilizando la vista **USER_SOURCE** o el comando **SQL DESCRIBE**.

Enlaces de interés...

En el siguiente enlace, encontrarás un vídeo en el que se podrás descubrir algunos ejemplos de uso de procedimientos que te permitirán ampliar tus conocimientos sobre estos: https://www.youtube.com/watch?v=_Gy8-hCA8a0

/ 3. Funciones

Son **subprogramas que guardan gran similitud con los procedimientos**, al punto de que pueden considerarse una categoría de ellos. Se utilizan principalmente para **realizar operaciones y cálculos necesarios para devolver un valor específico**. Todo lo relacionado con los parámetros, como se discutió en la sección anterior, es aplicable a las funciones. Además, las funciones pueden ser referenciadas desde otras partes del código o incluso desde SQL, aunque con ciertas limitaciones, las cuales se explican en el siguiente audio:

Audio 2. "Llamadas a funciones PL/SQL desde SQL"

https://on.soundcloud.com/yV2SWMGb3naKDCeeA

La **sintaxis** para declarar funciones es **casi idéntica a la de los procedimientos**, con algunas **diferencias clave**, como se muestra en la figura 5.

```
CREATE [OR REPLACE] FUNCTION nombreFuncion
[(variable1 [modelo] tipoDatos,
  variable2 [modelo] tipoDatos…)]
RETURN tipoDatos
{IS|AS}
Declaraciones
BEGIN …
```

Fig.5. Sintaxis de declaración de una función.

En este caso, se reemplaza la palabra **PROCEDURE** por **FUNCTION**, y, lo más importante, se introduce la cláusula **RETURN** para especificar el tipo de dato que devolverá la función.

PL/SQL también permite características comunes en otros lenguajes de programación, como la **sobrecarga**, donde pueden existir funciones con el mismo nombre, pero con diferentes números o tipos de parámetros, y la **recursividad**, en la que una función puede llamarse a sí misma. En la figura 6, podemos ver el ejemplo de un código recursivo.

```
CREATE OR REPLACE FUNCTION FACTORIAL (n NUMBER)
RETURN NUMBER IS
BEGIN
    IF (n <= 1) THEN
        RETURN 1;
    ELSE
        RETURN n * FACTORIAL(n - 1);
    END IF;
END;
```

Fig.6. Ejemplo de código recursivo.

Finalmente, si es necesario **eliminar una función**, se puede hacer utilizando el comando **DROP FUNCTION**, seguido del nombre de la función.

/ 4. Paquetes

Permiten agrupar funciones, procedimientos, tipos de datos y otros elementos, lo que facilita la organización y el mantenimiento de los programas.

La **declaración** de un paquete **se divide en dos secciones** principales: la **especificación** y el **cuerpo**. La **especificación** es donde se **declaran las variables, los cursores, las constantes, las excepciones, los procedimientos y las funciones** del paquete. En el **cuerpo** del paquete es donde se **escribe el código funcional**, **definiendo** los **procedimientos** y **funciones** previamente declarados en la especificación. Además, el cuerpo **puede contener variables y procedimientos privados**, los cuales solo pueden ser llamados dentro del propio paquete. La **sintaxis** básica de un paquete se muestra en la figura 7.

```
CREATE [OR REPLACE] PACKAGE nombrePaquete AS
[declaración de variables, constantes, cursores,
    excepciones, funciones y procedimientos]
END nombrePaquete;

CREATE [OR REPLACE] PACKAGE BODY nombrePaquete AS
[declaración privadas y cuerpo de los
 procedimientos y funciones]
END nombrePaquete;
```

Fig.7. Sintaxis de declaración de un paquete.

En la figura 8, se expone el **cuerpo** de la implementación de un paquete de prueba:

```
CREATE OR REPLACE PACKAGE BODY pruebaPaquete IS
    PROCEDURE procedimiento(contador NUMBER) IS
    BEGIN
        contador := nuevoContador;
    END procedimiento;

    FUNCTION obtenerContador IS
    BEGIN
        RETURN contador;
    END obtenerContador;
END pruebaPaquete;
```

Fig.8. Implementación de un paquete de prueba.

Para **llamar a una función o procedimiento que está dentro de un paquete desde fuera de este**, es necesario **anteponer el nombre del paquete al nombre de la función o procedimiento**. En el ejemplo de la imagen anterior, podríamos referenciarlo utilizando la sentencia '**pruebaPaquete.procedimiento(2)**'.

En el siguiente vídeo, se presentan ejemplos prácticos de procedimientos, funciones y paquetes:

Vídeo 1. "Procedimientos, funciones y paquetes en Oracle SQL Developer"

https://bit.ly/33D8yED

/ 5. Disparadores o triggers

Ahora exploraremos otro de los elementos clave en PL/SQL para la gestión de la base de datos: los disparadores.

5.1. Concepto de disparador

Un **disparador (*trigger* en inglés)** es un tipo de procedimiento que **se ejecuta automáticamente al realizar una operación DML en la base de datos**, a diferencia de las funciones y procedimientos que requieren ser llamados explícitamente.

Los **disparadores** se utilizan ampliamente en el entorno de las bases de datos para:

- **Rastrear** los cambios en las tablas e identificar al usuario que los realizó.
- **Automatizar** tareas de mantenimiento o administrativas.
- **Aplicar** reglas de integridad.

Es importante que el número de disparadores implementados sea el **estrictamente necesario**, evitando aquellos duplicados o recursivos, ya que pueden consumir una cantidad significativa de recursos.

5.2. Creación de disparadores

Al definir un disparador, es esencial considerar varios aspectos además de las acciones que debe realizar:

- **Instrucción que lo activará**: Como un INSERT, UPDATE o DELETE.
- **Momento en el que se ejecutará**: Ya sea antes (BEFORE), después (AFTER) o en lugar de la operación (INSTEAD OF).
- **Frecuencia de ejecución**: Puede activarse una sola vez por instrucción (disparador de instrucción) o una vez por cada fila afectada (disparador de fila).

Así, la **sintaxis** general se presenta en la figura 9.

```
CREATE [OR REPLACE] TRIGGER nombreDisparador
{BEFORE | AFTER | INSTEAD OF}
{INSERT [OR] UPDATE [OR] DELETE} ON Tabla
[REFERENCING OLD AS nomAntiguo NEW AS nomNuevo]
[FOR EACH ROW [WHEN (Condición de disparo)]]
BEGIN
   -- Bloque_de_código_del_Disparador
END nombreDisparador;
```

Fig.9. Sintaxis de declaración de un disparador.

Las cláusulas **REFERENCING** y **WHEN** son aplicables únicamente a los disparadores de fila. La cláusula **REFERENCING** permite asignar alias a los valores nuevos y antiguos, aunque su uso no es muy común.

En el siguiente vídeo, se presentan varios ejemplos de disparadores, destacando la diferencia entre los de instrucción y los de fila.

Vídeo 2. "Ejemplos de creación de disparadores"

https://bit.ly/36wLidv

/ 6. Explotación de disparadores: Restricciones, orden y gestión

En esta sección, abordaremos varios aspectos clave relacionados con la implementación y gestión de disparadores en PL/SQL, desde las restricciones que deben considerarse hasta el orden en que se ejecutan y cómo se pueden gestionar.

6.1. Restricciones en los disparadores

Al crear y utilizar disparadores, es fundamental tener en cuenta ciertas **limitaciones** específicas:

- Las operaciones **COMMIT, ROLLBACK, SAVEPOINT** o **SET TRANSACTION** están prohibidas dentro de un disparador.
- No se permite declarar variables de tipo **LONG** o **LONG RAW**.
- Tampoco está permitido hacer referencia a **procedimientos** o **funciones** que realicen **operaciones de tipo transacción**.

6.2. Orden de ejecución de los disparadores

Una misma tabla puede tener varios disparadores asociados, incluso de tipos diferentes, como **disparadores de instrucción** (se ejecutan **una vez por cada operación DML**) o **de fila** (se ejecutan **una vez por cada fila afectada**).

Dado que estos disparadores **no se ejecutan en paralelo**, es fundamental conocer el **orden de ejecución**, lo cual es esencial para un diseño adecuado. El orden de ejecución es el siguiente:

1. Disparadores de tipo instrucción con la cláusula **BEFORE**.
2. Disparadores de tipo fila con la cláusula **BEFORE**.
    - » Aquí, se ejecuta la sentencia que provoca el disparador.
3. Disparadores de tipo fila con la cláusula **AFTER**.
4. Disparadores de tipo instrucción con la cláusula **AFTER**.

6.3. Borrado y activación o desactivación de disparadores

Para **eliminar** un disparador se utiliza la sentencia **DROP TRIGGER**, como se muestra en la figura 10.

```
DROP TRIGGER nombreDisparador;
```

Fig.10. Sentencia de borrado de un disparador.

Asimismo, para **habilitar o deshabilitar un disparador**, se emplea la sentencia **ALTER TRIGGER**, como en la figura 11.

```
ALTER TRIGGER nombreDisparador {ENABLE | DISABLE};
```

Fig.11. Sentencia para habilitar o deshabilitar un disparador.

Finalmente, es posible **activar o desactivar todos los disparadores** asociados a una tabla específica utilizando la sentencia **ALTER TABLE**, como en la figura 12.

```
ALTER TABLE nombreTabla {ENABLE | DISABLE} ALL TRIGGERS;
```

Fig.12. Sentencia para controlar todos los disparadores de una tabla.

/ 7. Explotación de disparadores: Referencias y condicionales

En esta sección, exploraremos cómo utilizar referencias y cláusulas condicionales dentro de los disparadores en PL/SQL, herramientas esenciales para gestionar y controlar los cambios en las bases de datos de manera eficiente.

7.1. Referencias NEW y OLD

Cuando se emplea la sentencia **UPDATE** en una base de datos, los **valores actuales** de los registros **se reemplazan por nuevos valores**. Dentro de un **disparador**, las palabras clave **OLD** y **NEW** permiten acceder a los **valores antiguos y nuevos, respectivamente**, lo que facilita la **comparación y manipulación** de los datos **antes y después de la actualización**.

En la figura 13, se presenta un ejemplo de cómo utilizar estas referencias.

```
CREATE OR REPLACE TRIGGER auditoriaProductos
BEFORE UPDATE OF precio ON PRODUCTOS
FOR EACH ROW
WHEN (OLD.precio < NEW.precio)
BEGIN
    INSERT INTO AUDITORIA_PRODUCTOS
    VALUES(:OLD.precio, :NEW.precio, SYSDATE);
END;
```

Fig.13. Uso de las referencias NEW y OLD.

Este ejemplo asume que estamos realizando una auditoría sobre los precios de los productos en la tabla **PRODUCTOS**. Para registrar los cambios de precio, se ha creado una tabla **AUDITORIA_PRODUCTOS**.

Cada vez que se aumente el precio de un producto, el disparador agregará una nueva fila en **AUDITORIA_PRODUCTOS**.

7.2. Cláusulas IF INSERTING, IF UPDATING e IF DELETING

Estas cláusulas se emplean cuando un disparador puede activarse mediante diferentes sentencias DML (por ejemplo, un disparador que se ha creado con la instrucción INSERT OR DELETE OR UPDATE ON tabla).

Con estas cláusulas, se puede identificar la operación DML que disparó la ejecución del disparador, devolviendo **TRUE** cuando corresponde. Así, si el disparador se activa por una instrucción **INSERT**, la cláusula **IF INSERTING** devolverá **TRUE**.

En la figura 14 se muestra un ejemplo de cómo utilizar estas cláusulas:

```
CREATE OR REPLACE TRIGGER pruebaClausulas
BEFORE INSERT OR DELETE OR UPDATE OF campoPrueba ON tablaPrueba
FOR EACH ROW
BEGIN
    IF DELETING THEN
        --Instrucciones si el trigger se ejecuta al eliminar filas
    ELSIF INSERTING THEN
        --Instrucciones si el trigger se ejecuta al insertar filas
    ELSE
        --Instrucciones si el trigger se ejecuta al modificar filas
    END IF;
END;
```

Fig.14. Ejemplo de uso de cláusulas IF INSERTING, IF DELETING e IF UPDATING.

8. Explotación de los disparadores: Tipos especiales e información adicional

En esta sección exploraremos los disparadores de tipo **INSTEAD OF**, así como la manera de obtener información sobre los disparadores existentes en la base de datos.

8.1. Disparadores de tipo INSTEAD OF

Se utilizan exclusivamente en vistas. Recordemos que una vista es una consulta **SELECT** almacenada en la base de datos, diseñada principalmente para mostrar datos, aunque también se puede utilizar para modificarlos.

Estos disparadores **son útiles cuando se desea actualizar una vista que hace referencia a varias tablas**. Por ejemplo, en la vista definida en la figura 15, se selecciona información de productos en el almacén, combinando datos de las tablas '**productos**' y '**almacén**'.

```
CREATE VIEW infoProductos (modelo, precio, cantidad)
AS SELECT p.modelo, p.precio, a.cantidad
     FROM productos p
     JOIN almacen a ON p.modelo = a.modelo
```

Fig.15. Código de una vista de ejemplo.

Si intentásemos añadir un nuevo elemento a esta vista, el sistema generaría un error. La solución sería insertar primero los datos necesarios en cada una de las tablas involucradas (en este caso, '**productos**' y '**almacén**'), lo cual se puede lograr mediante un disparador **INSTEAD OF** como el de la figura 16.

```
CREATE OR REPLACE TRIGGER insertarProducto
INSTEAD OF INSERT ON infoProductos
BEGIN
    INSERT INTO productos(modelo,precio)
    VALUES (:NEW.modelo, :NEW.precio);
    INSERT INTO  almacen (modelo,precio,cantidad)
    VALUES (:NEW.modelo, :NEW.precio, :NEW.cantidad);
END;
```

Fig.16. Disparador de tipo INSTEAD OF.

8.2. Información sobre disparadores

La vista **USER_TRIGGERS** proporciona información detallada sobre los disparadores existentes en la base de datos. Se puede obtener una descripción de esta vista utilizando el comando **DESC USER_TRIGGERS**, como se muestra en la figura 17.

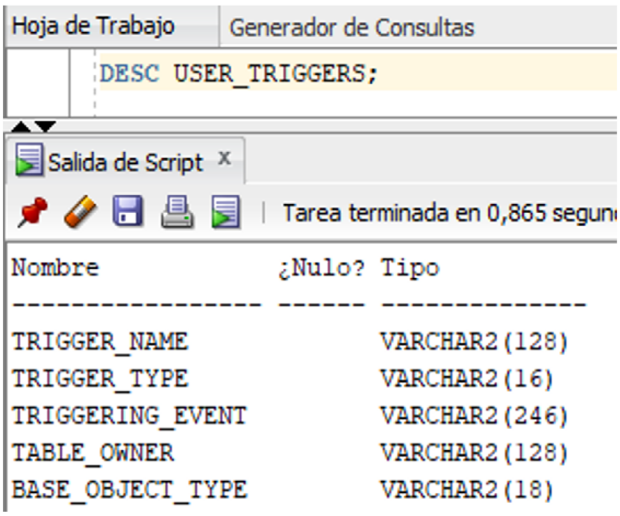


Fig.17. Vista USER_TRIGGER con los disparadores existentes.

/ 9. Automatización de la ejecución de scripts

Automatizar la ejecución de scripts en PL/SQL es clave para gestionar eficientemente las bases de datos Oracle. Esta automatización optimiza la eficiencia, reduce errores humanos y garantiza la consistencia en tareas repetitivas. Existen varias opciones para ello, veremos ahora algunas de ellas y exploraremos detenidamente la más utilizada en la siguiente pantalla.

9.1. Oracle Enterprise Manager (OEM)

Es una solución completa de gestión de bases de datos que incluye capacidades para programar y monitorizar la **ejecución de scripts PL/SQL**. Con una interfaz gráfica intuitiva, permite gestionar tareas y asegurar su correcta ejecución sin necesidad de escribir comandos manualmente. Esta herramienta es ideal para administradores que buscan centralizar y simplificar la gestión de sus bases de datos.

9.2. DBMS_JOB

Es un paquete PL/SQL que permite la programación de trabajos en Oracle. Aunque ha sido en gran parte reemplazado por **Oracle Scheduler**, sigue siendo una opción válida para la ejecución automática de tareas dentro de la base de datos. Es especialmente útil para tareas más simples y menos demandantes en términos de programación y recursos.

9.3. Shell Scripts con SQL*Plus

SQL*Plus es una herramienta de línea de comandos diseñada para **ejecutar scripts SQL y PL/SQL**. Es posible automatizar estos procesos creando un script en *shell*, como *bash*, que ejecute el comando 'sqlplus' automáticamente.

En la figura 18 se muestra un ejemplo de un script de shell.

```
#!/bin/bash
sqlplus username/password@database <<EOF
@/path/to/your/script.sql
EXIT;
EOF
```

Fig.18. Script de shell.

Este script puede configurarse para ejecutarse automáticamente utilizando herramientas como 'Cron' en sistemas Unix/Linux.

9.4. Cron Jobs (Linux/Unix)

Podemos utilizar el sistema de CronJobs en Linux/Unix para programar la ejecución automática de scripts PL/SQL a intervalos regulares.

En la figura 19 se muestra un ejemplo de una entrada del *crontab* correspondiente a una tarea que se ejecutará todos los días a las 5:00 AM.

```
0 5 * * * /scripts/job.sh
```

Fig.19. Ejemplo de tarea cron.

/ 10. Automatización de la ejecución de scripts: Oracle Scheduler

Oracle Scheduler es una herramienta avanzada en Oracle Database que permite programar y automatizar la ejecución de trabajos. Con esta funcionalidad, es posible planificar la ejecución de scripts PL/SQL en horarios específicos o de forma recurrente.

10.1. Creación de jobs

La creación de un *job* se realiza con el procedimiento **DBMS_SCHEDULER.create_job**. Algunos parámetros clave son:

- **job_name**: El nombre único del job dentro de la base de datos.
- **job_type**: El tipo de job, como **PLSQL_BLOCK** o **STORED_PROCEDURE**.
- **job_action**: La acción a ejecutar, que puede ser un bloque PL/SQL, una llamada a un procedimiento almacenado o un comando de shell.
- **start_date**: La fecha y hora de inicio del job.
- **repeat_interval**: El intervalo de repetición, definido por una expresión de calendario que establece la frecuencia del job.
- **enabled**: Este parámetro controla si el job está habilitado para ejecutarse.

En la figura 20 se muestra un ejemplo de cómo definir un job utilizando Oracle Scheduler.

```
BEGIN
  DBMS_SCHEDULER.create_job (
    job_name        => 'mi_job',
    job_type        => 'PLSQL_BLOCK',
    job_action      => 'BEGIN mi_procedimiento(); END;',
    start_date      => SYSTIMESTAMP,
    repeat_interval => 'FREQ=DAILY; BYHOUR=10;',
    enabled         => TRUE
  );
END;
```

Fig.20. Creación de job con Scheduler.

10.2. Modificación y eliminación de jobs

Para modificar un job, se utiliza el procedimiento **DBMS_SCHEDULER.set_attribute**, que permite ajustar parámetros como el intervalo de repetición o la acción a realizar.

Por otro lado, para eliminar un job de forma definitiva, se emplea el procedimiento **DBMS_SCHEDULER.drop_job**.

Investigamos...

Además de las propiedades vistas en Oracle Scheduler para programar los jobs, existe el concepto de 'ventana'. Investiga sobre él y practica su uso para mejorar tus conocimientos.

10.3. Comandos adicionales

Además de la creación, modificación y eliminación de jobs, **Oracle Scheduler** ofrece comandos adicionales para gestionar los mismos:

- **DBMS_SCHEDULER.enable** y **DBMS_SCHEDULER.disable**: Permiten habilitar o deshabilitar jobs sin eliminarlos.
- **DBMS_SCHEDULER.run_job**: Permite ejecutar un job de manera manual en cualquier momento.
- **DBMS_SCHEDULER.stop_job**: Detiene la ejecución de un job que se está ejecutando en ese momento.

/ 11. API de acceso a bases de datos

En trabajos relacionados con el mantenimiento y explotación de bases de datos, es muy habitual (especialmente en la actualidad) que sea necesario implementar accesos a las bases de datos desde lenguajes de programación externos, con el fin de poder obtener información de estas desde diferentes aplicaciones o funcionalidades, que a su vez se han desarrollado en otros sistemas y lenguajes.

- **Oracle APEX (Application Express)**: Oracle APEX es una plataforma de desarrollo rápido para crear aplicaciones web y móviles con interfaces gráficas avanzadas. Ofrece un entorno visual basado en el navegador con herramientas para diseñar páginas, aplicar temas y crear aplicaciones responsivas.
- **Oracle JDeveloper**: Es un IDE que admite lenguajes como Java, XML, SQL, y HTML. Utiliza Oracle ADF, que incluye componentes JSF para crear interfaces web complejas, y simplifica el desarrollo de aplicaciones empresariales con un editor visual e integración con bases de datos Oracle.

- **Oracle Visual Builder**: Es una plataforma en la nube que permite desarrollar aplicaciones web y móviles utilizando un enfoque visual *low-code*. Está diseñada para integrarse perfectamente con otros servicios de Oracle Cloud, facilitando así el despliegue directo de las aplicaciones en la nube.

- **Oracle Forms**: Es una herramienta tradicional para desarrollar aplicaciones empresariales con interfaces gráficas, especialmente en entornos cliente-servidor. También permite migrar estas aplicaciones a un entorno web, manteniendo una sólida integración con bases de datos Oracle.

- **Oracle API Catalog (OAC)**: Permite gestionar API y servicios web, facilitando a las organizaciones catalogar, documentar y centralizar sus API para facilitar su descubrimiento y reutilización. Es clave en entornos que integran múltiples servicios, asegurando un control eficiente de las API utilizadas.

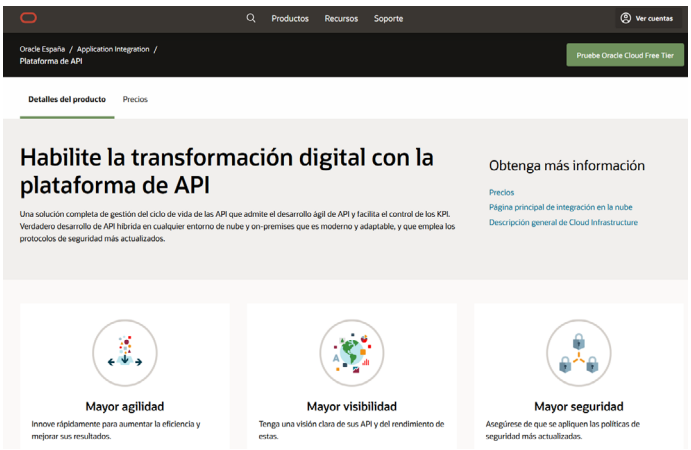


Fig.21. Plataforma de API Oracle.

Enlaces de interés...

A través del siguiente enlace, podrás acceder a la plataforma de formación para desarrolladores de Oracle, que ofrece ejemplos y guías para el desarrollo, lo que puede ser de gran ayuda para profundizar en este campo: https://developer.oracle.com/es/api/

/ 12. Caso práctico 1: "Procedimientos y funciones más complejas"

Planteamiento: Aplicaremos los conceptos teóricos aprendidos en este tema para implementar funciones y procedimientos más complejos que los abordados en temas anteriores.

Nudo: Para ello, crearemos una función que determine si el salario de un empleado (recibido como parámetro) es superior o inferior a la media de los salarios de los empleados del mismo departamento. Es importante considerar que la tabla EMPLEADOS incluye una columna que hace referencia al departamento.

Desenlace:

```
CREATE FUNCTION comprobarSalario(a NUMBER) RETURN BOOLEAN IS
    id_dpto EMPLEADOS.id_departamento%TYPE;
    id_empl EMPLEADOS.id_empleado%TYPE;
    sal EMPLEADOS.salario%TYPE;
    m_sal EMPLEADO.salario%TYPE;
BEGIN
    SELECT salario,id_empleado,id_departamento INTO sal,id_empl, id_dpto FROM EMPLEADOS
    WHERE id_empl = a;
    SELECT AVG(SALARIO) INTO m_sal FROM EMPLEADOS
    WHERE id_departamento = id_dpto;
    IF sal > m_sal THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN NULL;
END;
```

Fig.22. Función para la comparación de salarios.

La figura 22 muestra un ejemplo de código que podría ser el resultado de este caso práctico.

/ 13. Caso práctico 2: "Creación de disparadores"

Planteamiento: En esta ocasión, pondremos en acción la teoría estudiada a lo largo del tema. La mejor manera de aprender a programar es practicando, por lo que vamos a trabajar en la creación de disparadores.

Nudo: Para ello, definiremos un disparador que se activará automáticamente cuando se inserte o modifique una fila en la tabla EMPLEADOS. Dado que cada empleado debe pertenecer a un departamento, es importante considerar esta restricción en el cuerpo del disparador.

Desenlace:

```
CREATE OR REPLACE TRIGGER control_empleados
BEFORE INSERT OR UPDATE OF empleados
FOR EACH ROW
BEGIN
    IF (:new.departamento IS NULL) THEN
    RAISE_APPLICATION_ERROR(-20201, 'Todo
    empleado debe pertenecer a un departamento');
    END IF;
END;
```

Fig.23. Crear un disparador.

El código que es necesario implementar para crear este disparador es el que se muestra en la figura 23.

/ 14. Resumen y resolución del caso práctico de la unidad

En este tema, hemos completado el estudio de PL/SQL, comenzando con los procedimientos, sus similitudes con las funciones y el uso de paquetes, acompañados de ejemplos prácticos. Luego, hemos analizado los disparadores, esenciales para la gestión de bases de datos, revisando su creación, sus características y aplicaciones.

Además, hemos explorado las formas de automatizar la ejecución de scripts en Oracle, antes de concluir con las API de acceso a bases de datos desde lenguajes externos, enfocándonos en las opciones que ofrece Oracle para el desarrollo y la gestión.

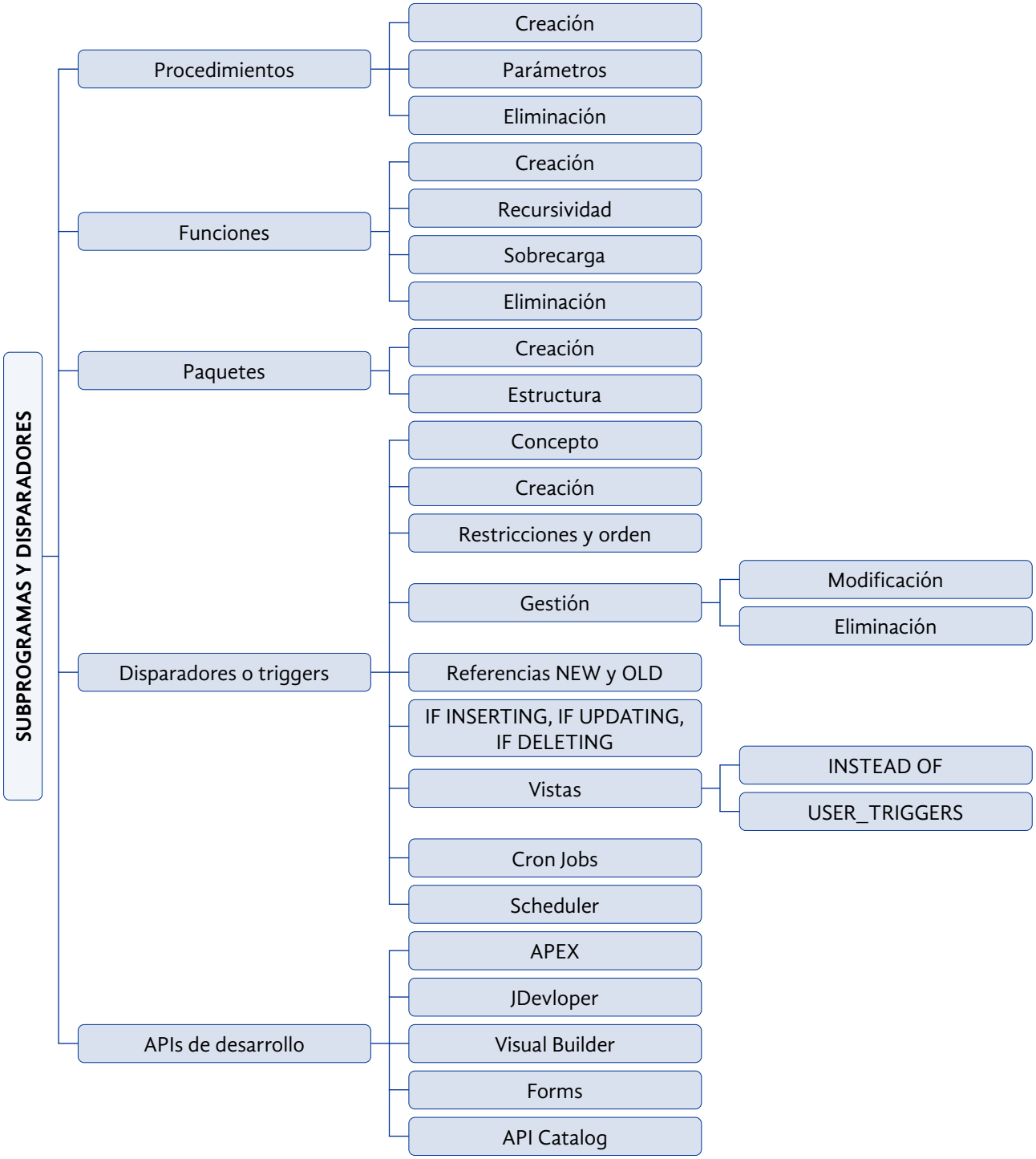


Fig.24. Esquema resumen del tema.

Resolución del caso práctico de la unidad

En el audio del caso práctico inicial se planteaban varias preguntas que abordaremos a continuación.

Las primeras cuestiones que analizaremos son las siguientes:

¿Podríamos realizar algún código PL/SQL que le pueda ayudar aún más en la gestión y mantenimiento de su base de datos, y que pueda ser utilizado cuándo y dónde se necesite? ¿De qué forma podríamos lograr algo así?

Gracias a PL/SQL, es posible resolver prácticamente cualquier funcionalidad que se necesite para utilizar y aprovechar la base de datos, además de automatizar el código requerido. A través de procedimientos, funciones y paquetes, podemos desarrollar código más complejo que puede ser llamado desde otras partes del programa o incluso desde sentencias SQL.

Por lo tanto, el uso de estas estructuras de código ofrece un amplio abanico de posibilidades.

La otra pregunta que planteamos es la siguiente:

¿Podríamos implementar alguna funcionalidad que permita detectar de manera precisa y eficaz los cambios realizados en cualquier tabla de la base de datos y quién lo ha llevado a cabo?

Como hemos visto a lo largo de este tema, esta funcionalidad puede implementarse utilizando disparadores, los cuales resultan muy útiles para el mantenimiento y la optimización de sistemas de bases de datos.

/ 15. Bibliografía

Connolly, T. y Begg, C. (2005). *Sistemas de bases de datos* (5.ª ed.). Madrid, España. Addison-Wesley.

Elmasri, R. y Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison-Wesley.

López Montalbán, Iván y De Castro, M. (2014). *Gestión de bases de datos: Técnico Superior en Administración de Sistemas Informáticos en Red*. Madrid. Garceta D.L.

Oppel, A. (2009). *Databases. A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España. Alfaomega.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos*. Paraninfo.

Urman S., Hardman, R. y McLaughlin, M. (2004). *Oracle database 10g. PL/SQL Programming*. Oracle Press.